

Food Ordering System - IT096IIU Lab 01 (Go Foundation)

NguyenChiBao-ITCSIU24008

Đổng Đức Dũng-ITCSIU24023

Technical report and user guide for Lab 01 of IT096IIU - Netcentric Programming, International University. The system simulates a minimal food-ordering workflow for an on-campus dining outlet. It supports practical study of data design, Go slices and maps, automated testing, and adaptability to changing requirements.

1. Introduction

This laboratory exercise follows the five-stage workflow prescribed in the assignment: THINK → BUILD → VERIFY → ADAPT → GROUP CHECK. Its guiding principle is “Think together. Build with AI. Verify together. Own the result.” Each stage is documented separately in the repository, distinguishing (a) requirements specifications, (b) implementation source code, and (c) evidence of verification and adaptation. This separation supports assessment against individual learning objectives.

The functional scope follows the scenario specified in the assignment:

1. Display available food items.
2. Search for food items by name.
3. Add food items to an order.
4. Reject invalid quantities.
5. Reject unavailable food items.
6. Calculate the order subtotal.
7. Display an order summary.
8. Apply a discount following a requirements change.

2. Project Structure

File in Lab1/	Purpose
CLAUDE.md	Laboratory and homework requirements (Sections 1-9), with supplementary clarifications.
AGENTS.md	Project guidance for AI assistants.
README.md	The present document.
VERIFY.md	Stage 4 - VERIFY: test cases, defect injection, and AI review.
ADAPT.md	Stage 5 - ADAPT: discount requirements, tests, and reflection.
GROUP_CHECK.md	Stage 6 - GROUP CHECK: preparation for instructor questions.
HOMEWORK.md	Section 9 - HOMEWORK: customer and order types, the revised formula, and AI review.
CODE_QA.md	Detailed questions and answers concerning the current code, logic, and design.
go.mod	Go module declaration: module foodordering.
main.go	Complete implementation: structures, slices, maps, business logic, and CLI.
main_test.go	14 test functions, including 12 homework combination subtests.
IT096IIU_Lab01_Food_Ordering_Student (1).pdf	Original assignment provided by the instructor.

Organizational principle: source code (main.go and main_test.go) is separated from evidence and design rationale (VERIFY.md, ADAPT.md, and GROUP_CHECK.md). This arrangement reflects the assessment criteria in Section 8 of CLAUDE.md, which evaluate evidence from each stage rather than only the final product.

3. System Requirements

go.mod declares go 1.27.1; the verification environment on 6 October 2026 used go1.27.1 windows/amd64. Use a toolchain that satisfies the version declared in the module. Go 1.21 should not be described as the minimum version for the current configuration.

The application has no external dependencies. It uses only the Go standard library: bufio, fmt, os, strconv, and strings.

Check the installed Go version:

```
go version
```

4. Usage

4.1 Building the Application

```
cd Lab1  
go build -o foodordering.exe .
```

This command compiles main.go into the executable foodordering.exe. Because go.mod declares the foodordering module, manual GOPATH configuration is unnecessary.

4.2 Running the Application

```
./foodordering.exe
```

The application presents a repeating command-line menu:

```
1. Display available food  
2. Search food by name  
3. Add food to order  
4. Show order summary  
5. Set customer type (Regular/Member)  
6. Set order type (Pickup/Delivery)  
7. Exit  
Choose an option:
```

Option 1 - Display available food. Lists every menu item whose Available field is true, including its name, category, and price. Unavailable items, such as “Fried Rice” and “Cheesecake” in the sample data, are excluded.

Option 2 - Search food by name. Accepts an exact, case-sensitive food name. If the item exists, its complete information, including availability, is displayed. Otherwise, the application prints “Food not found” without producing a runtime error.

Option 3 - Add food to order. Accepts a food name followed by a quantity. Validation proceeds in this order: item existence → item availability → quantity validity (> 0). If any condition fails, the application prints an "Error: ..." message identifying the cause and leaves the order unchanged.

Option 4 - Show order summary. Displays each order line (name, quantity, and line total), followed by the customer type, order type, Subtotal, Discount (the ADAPT rule: 10% when the subtotal is at least 300,000 VND), Member Discount (5% for Member customers), Delivery Fee, and Final Total.

Option 5 - Set customer type. Accepts Regular or Member as exact, case-sensitive values. Members receive a 5% discount on the food subtotal. Other input is rejected without modifying the order.

Option 6 - Set order type. Accepts Pickup or Delivery. Delivery adds a fee of 30,000 VND after all discounts have been deducted. Other input is rejected without modifying the order.

Option 7 - Exit. Terminates the application.

A new order defaults to Regular + Pickup, with no member discount or delivery surcharge. The 10% volume discount still applies when the subtotal reaches 300,000 VND. HOMEWORK.md explains the formula and the combination of the two discounts.

4.3 Example Session

```
Choose an option: 3
Enter food name: Chicken Rice
Enter quantity: 7
Added to order.
Choose an option: 3
Enter food name: Milk Tea
Enter quantity: 2
Added to order.
Choose an option: 4
Order summary:
- Chicken Rice x7 = 315000 VND
- Milk Tea x2 = 50000 VND
Customer: Regular | Order: Pickup
Subtotal: 365000
Discount: 36500
Member Discount: 0
Delivery Fee: 0
Final Total: 328500
```

For the same order, selecting Member + Delivery through Options 5 and 6 produces a Final Total of 340250: 365000 - 36500 - 18250 + 30000.

4.4 Running Automated Tests

```
go test -count=1 -v ./...
go vet ./...
```

The test command reports individual test cases and their PASS/FAIL results. Use -run to execute a specific test, for example:

```
go test -v -run TestDiscountAtThreshold ./...
```

VERIFY.md provides detailed test results and documents the defect-injection and correction procedure.

5. Data Design

5.1 FoodItem

```
type FoodItem struct {  
    Name      string  
    Price     float64  
    Category  string  
    Available bool  
}
```

These four fields conform to the assignment scenario. Price uses float64, following the representation in the assignment example and facilitating multiplication by the rates 0.10 and 0.05. This is a simple laboratory design choice, rather than a requirement for percentage calculations. A production system could store VND amounts as int64 and define explicit rounding rules. The current code applies %.0f only when printing and does not implement a separate monetary rounding policy.

5.2 OrderLine and Order

```
type OrderLine struct {  
    Item      FoodItem  
    Quantity  int  
}  
  
type Order struct {  
    Lines     []OrderLine  
    Customer  CustomerType  
    Type      OrderType  
}
```

Customer and Type were introduced in the homework extension (Section 9). They belong to Order rather than OrderLine because customer classification and fulfillment method are properties of the entire order; see Section 9.3 of HOMEWORK.md.

The principal THINK-stage design decision concerns the assignment example, which proposes two parallel slices, []FoodItem and []int (quantities), passed separately to CalculateSubtotal. This design was evaluated and rejected because independently maintained slices can differ in length unless every insertion and deletion is carefully synchronized. Such a mismatch is a preventable class of error.

The selected design combines each FoodItem and its Quantity in one OrderLine structure. Order therefore maintains a single []OrderLine slice. Each element contains both the item and its quantity, making a length mismatch between separate lists impossible. The data structure enforces this invariant, rather than relying on a manually maintained programming convention.

5.3 Slices and Maps

The system uses both structures for distinct access requirements:

Structure	Variable	Role
Slice	menu []FoodItem	Primary source of truth. Preserves order and supports complete traversal when displaying available food.
Map	menuByName map[string]FoodItem	Name-based lookup index for searches and add-to-order validation. Constructed once from menu at startup.

Separating sequential traversal through a slice from key-based lookup through a map reflects two application access patterns. Displaying all items requires O(n) traversal, whereas retrieving an item by name has average O(1) map lookup complexity, compared with O(n) for a repeated slice search.

6. Business Logic: Principal Functions

Function	Responsibility
buildMenuByName	Constructs the lookup map from the menu slice.
DisplayAvailableFood	Traverses the slice and prints items whose Available field is true.
SearchFoodByName	Looks up an item in the map and returns (FoodItem, bool), following the Go comma-ok idiom. An unsuccessful lookup does not cause a panic.
IsValidQuantity	Enforces the quantity rule: only quantities greater than zero are valid.

Function	Responsibility
AddFoodToOrder	Performs all validation (item not found, unavailable item, and invalid quantity) before modifying the order. Returns a descriptive error on failure.
CalculateSubtotal	Sums Price × Quantity across all OrderLine entries.
CalculateDiscount	Applies the ADAPT rule: a 10% volume discount when subtotal ≥ 300,000 VND; otherwise zero.
CalculateCustomerDiscount	Homework extension: 5% of the food subtotal for Member customers; otherwise zero.
CalculateDeliveryFee	Homework extension: 30,000 VND for Delivery orders; otherwise zero.
CalculateFinalTotal	Homework extension: the single location defining Subtotal - VolumeDiscount - MemberDiscount + DeliveryFee.
DisplayOrderSummary	Combines the calculation functions to display order lines, Subtotal, both discounts, Delivery Fee, and Final Total.

Separation of concerns is applied consistently. Validation resides in AddFoodToOrder and IsValidQuantity; calculations reside in CalculateSubtotal and CalculateDiscount; presentation resides in DisplayAvailableFood and DisplayOrderSummary; input/output orchestration resides separately in main(). Consequently, business logic can be tested with go test without simulating command-line input; see main_test.go.

7. Related Documents

Document	Contents
CLAUDE.md	Requirements in Sections 1-9, with clarification of discount combination and the homework test matrix.
VERIFY.md	VERIFY-stage results for seven required test cases; defect injection → identification → explanation → correction → retesting; and an AI review of AddFoodToOrder.
ADAPT.md	ADAPT-stage changes for the discount rule, three threshold tests (below, at, and above), and reflection on design extensibility.

Document	Contents
GROUP_CHECK.md	Preparation for eight potential instructor challenges: explaining structures, slices/maps, functions, and edge cases; fixing a defect; adding category search; explaining AI-generated code; and changing the discount rule.
HOMEWORK.md	Section 9: Regular/Member customer types, Pickup/Delivery order types, discount-combination policy, 12 combination test cases, and an AI review of CalculateFinalTotal.
CODE_QA.md	Code-based defense questions and answers covering calculation traces, pointers, slices/maps, validation, tests, and design limitations.

8. Scope Notes

The homework extension has been implemented and documented in HOMEWORK.md. Subtotal, volume-discount, and add-item functions remain separate from customer-discount and delivery-fee functions. DisplayOrderSummary and main integrate the new functionality. The current verification confirms that the ten VERIFY/ADAPT tests still pass. However, the directory has no Git history, so these results cannot establish that the original functions have never been modified.

SearchFoodByCategory, discussed in Section 6.6 of GROUP_CHECK.md, has not been added to main.go. It is a prepared example for the potential GROUP CHECK challenge “Add search by category,” rather than a mandatory BUILD or homework requirement.

9. Alignment of the Homework Requirements with the Current Project

9.1 Conclusion and Differences between the Two Requirements Documents

The project satisfies the homework functionality and technical-evidence requirements in CLAUDE.md, with an explicitly documented cumulative-discount policy. Nevertheless, the current formula should not be described as identical to the standalone formula in Section 9 of the assignment PDF.

Pages 5-6 of the assignment PDF specify Regular/Member, Pickup/Delivery, a 5% member discount on the food subtotal, and Final Total = Subtotal - Customer Discount + Delivery Fee. They do not explicitly state whether the 10% ADAPT discount should be retained or combined with the homework discount.

Section 9 of CLAUDE.md additionally requires clarification of discount combination and testing of all four combinations at three subtotal levels. This is a project-document clarification, rather than wording reproduced verbatim from the assignment PDF. The code retains both discounts and calculates each from the original subtotal:

```

S = sum of Price × Quantity
V = S × 10% if S >= 300000; otherwise 0
C = S × 5% if Customer == Member; otherwise 0
F = 30000 if Type == Delivery; otherwise 0
Final Total = S - V - C + F

```

Regular customers receive no customer discount, but remain eligible for the volume discount at the specified threshold. At a subtotal of 300,000 VND, Member + Delivery yields 285,000 VND under the current code. Applying only the homework formula in the assignment PDF, without the volume discount, yields 315,000 VND. The cumulative-discount policy should therefore be explained during submission and oral defense. If the instructor requires the homework rule to replace ADAPT, the formula and test expectations must be revised accordingly.

9.2 Checklist and Supporting Evidence

Requirement	Project evidence	Assessment
Regular and Member; 5% of food subtotal	CustomerType, Order.Customer, CalculateCustomerDiscount	Satisfied
Pickup: 0; Delivery: 30,000 VND	OrderType, Order.Type, CalculateDeliveryFee	Satisfied
Member discount does not reduce delivery fee	Fee added separately from both discounts; TestMemberDiscountDoesNotReduceDeliveryFee	Satisfied
Explain combination of ADAPT and member discounts	Comment in main.go, HOMEWORK.md Section 9.2, and the formula above	Satisfied under the CLAUDE.md clarification
Summary displays discounts, fee, and final total	DisplayOrderSummary: Discount, Member Discount, Delivery Fee, Final Total	Satisfied; Discount denotes the volume discount

Requirement	Project evidence	Assessment
Four combinations × below/at/above 300,000	12 subtests in TestFinalTotalCombinations	Satisfied
Updated source code	main.go and go.mod	Available
Test cases and results	main_test.go, HOMEWORK.md Section 9.4, and the run reported below	Available
Brief design note	HOMEWORK.md Sections 9.2-9.3; README Sections 5-6	Available
One AI review	HOMEWORK.md Section 9.5; current review in Section 9.4 below	Review provided
Explanation of requirements changes	HOMEWORK.md Section 9.6	Available
Both members understand the solution	Each member must independently explain the solution and perform challenges	Cannot be confirmed solely by reading files or running tests

9.3 Verification Results: 6 October 2026

The following checks were performed on the current source using Go 1.27.1, without cached test results:

Check	Result
go test -count=1 -v ./...	PASS: 14 top-level test functions, including 12 combination subtests.
go vet ./...	PASS; no diagnostics.
Build source into a temporary executable	PASS.
CLI: Chicken Rice × 7 + Milk Tea × 2; Member + Delivery	Subtotal 365,000; volume discount 36,500; member discount 18,250; fee 30,000; final total 340,250.

Check	Result
Replace >= with > in a temporary source copy	TestDiscountAtThreshold and all four threshold subtests failed as expected. The primary source was unchanged.

Tested Final Total matrix (VND):

Subtotal	Regular + Pickup	Regular + Delivery	Member + Pickup	Member + Delivery
200,000	200,000	230,000	190,000	220,000
300,000	270,000	300,000	255,000	285,000
400,000	360,000	390,000	340,000	370,000

Counting convention: HOMEWORK.md reports “25/25 tests” by combining 13 other tests with 12 child cases. Actual Go output contains 14 test functions and 12 subtests, with one function serving as the parent of the 12 subtests. This explicit counting convention is preferable.

9.4 Current AI Review: CalculateFinalTotal (without Rewriting)

The function calculates the subtotal once, uses the same original subtotal for both discounts, and adds the delivery fee separately. Its logic is correct under the project’s cumulative-discount policy, and the 12 combination cases cover the corresponding scenarios. Decomposition into small functions helps isolate errors in subtotal, discount, or fee calculations without running the CLI.

The following limitations should be understood when explaining the code:

Named string types prevent confusion between typed categories, but do not restrict values as a closed enumeration would: `CustomerType("VIP")` remains type-valid. The CLI rejects invalid input; calculation functions assign zero discount to customer types other than Member and zero fee to order types other than Delivery.

Because fields are exported, callers can construct `OrderLine` entries with negative quantities or prices while bypassing `AddFoodToOrder`. Monetary calculations assume that the order has been constructed validly.

The CLI currently ignores I/O errors from `ReadString`. If standard input ends without selecting Option 7, the loop may repeatedly print “Invalid option.” The normal interactive workflow includes Exit; EOF handling is an area for improvement.

The map stores copies of `FoodItem` and is constructed only once. Runtime menu changes would require map synchronization. The current implementation uses a static menu.

The summary and CLI lack automated tests; the reported CLI session verifies only one scenario. Likewise, float64 and %.0f do not constitute an explicit monetary rounding policy.

These observations identify limitations and potential improvements, rather than missing homework functionality within a valid usage workflow. This review updates documentation only and does not alter business logic.

9.5 Submission Preparation

Submit main.go, go.mod, main_test.go, HOMEWORK.md, and README to provide source code, tests, a design note, an AI review, and an explanation of the requirements changes. CODE_QA.md supports practice in explaining the implementation; it does not replace each member's independent understanding.

VERIFY.md, ADAPT.md, and GROUP_CHECK.md primarily document laboratory stages preceding the homework extension. Some descriptions of Order, the CLI, and the summary in those files refer to earlier versions. README.docx has also not been synchronized with the Markdown README in this update.